Homework: Library Management Setup

1. Create and run a Laravel project named library-management.

I initialized a new Laravel project named `library-management` using Composer and started the local development server:

```
composer create-project laravel/laravel library-management cd library-management php artisan serve
```

The application runs locally on `http://127.0.0.1:8000`. Below is the screenshot of the Laravel welcome page running in the browser:

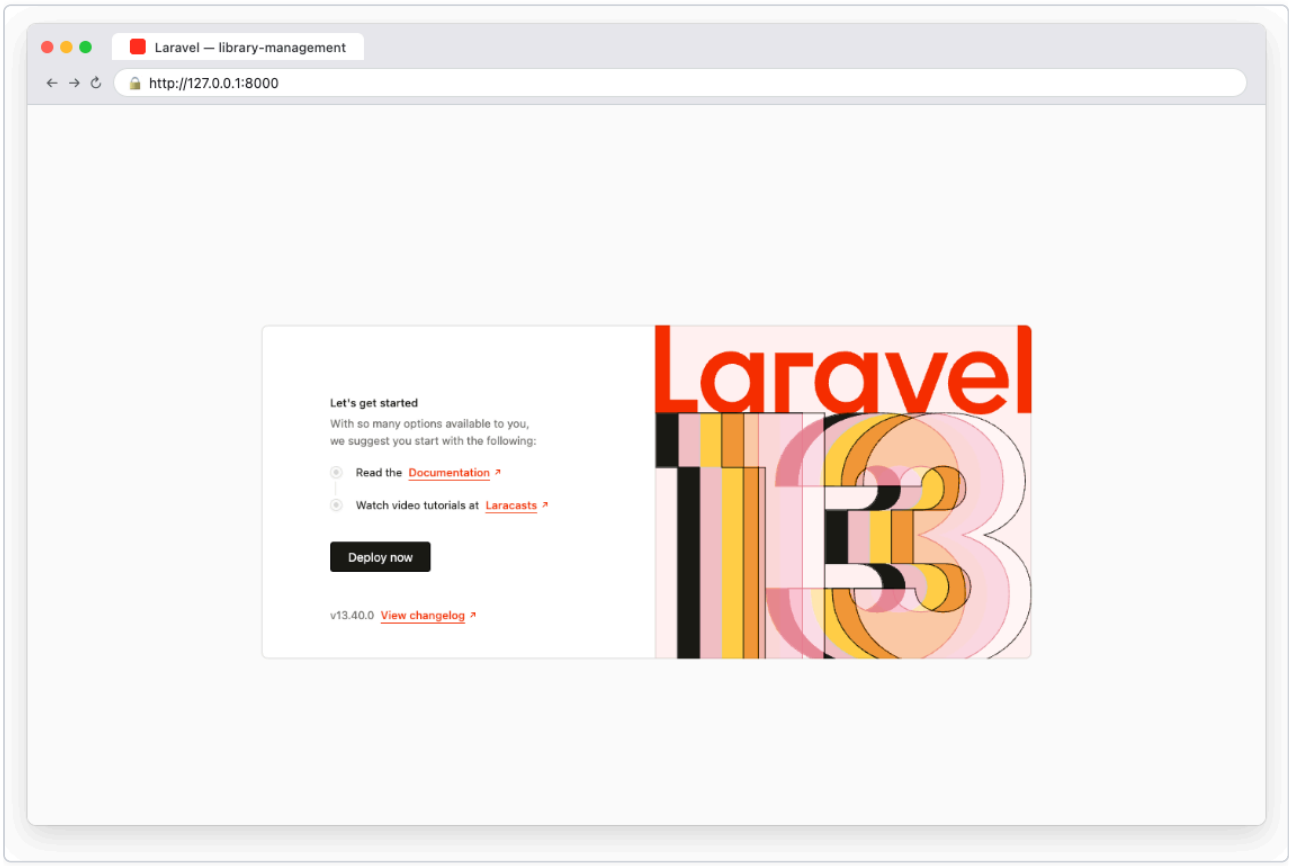


Figure 1: Laravel welcome page running locally on port 8000 (http://127.0.0.1:8000).

2. Briefly explain the purpose of app, routes, resources, database, and .env.

`app/` **Core Application Logic:** Contains the main PHP code for the application, including Eloquent models (such as `Book.php`, `Member.php`, and `Borrowing.php`) that map to database tables, HTTP controllers that process user requests and return responses, and middleware that filters HTTP requests (like authentication).

`routes/` **Route Definitions:** Defines all URL endpoints for the application. The primary file is `routes/web.php`, which handles web browser requests with session state and CSRF token protection. It also includes `routes/api.php` for stateless REST API routes and `routes/console.php` for Artisan CLI commands.

`resources/` **Presentation Layer & Frontend Assets:** Contains uncompiled frontend files, including Blade templates in `resources/views/` (which render the HTML pages users interact with), raw CSS/SCSS stylesheets, JavaScript files, and localization dictionaries.

`database/` **Database Management & Migrations:** Contains everything needed to manage database schemas. It includes `migrations/` (which define table structures in PHP code so they can be version-controlled), `seeders/` (to populate tables with sample data), and `factories/` (to generate fake test records).

`.env` **Environment Configuration:** Stores environment-specific variables and sensitive credentials, such as database connection parameters (`DB_CONNECTION=pgsql`, `DB_DATABASE=library_management`), application key, and mail settings, keeping them secure and separate from source code.

3. Create a PostgreSQL database named library_management.

I created a new PostgreSQL database named `library_management` and updated the Laravel `.env` file:

```
DB_CONNECTION=pgsql DB_HOST=127.0.0.1 DB_PORT=5432 DB_DATABASE=library_management
DB_USERNAME=leangkhai DB_PASSWORD=
```

Then I executed `php artisan migrate` to run all migrations and build the tables in PostgreSQL. Below is the terminal screenshot verifying the database and tables:

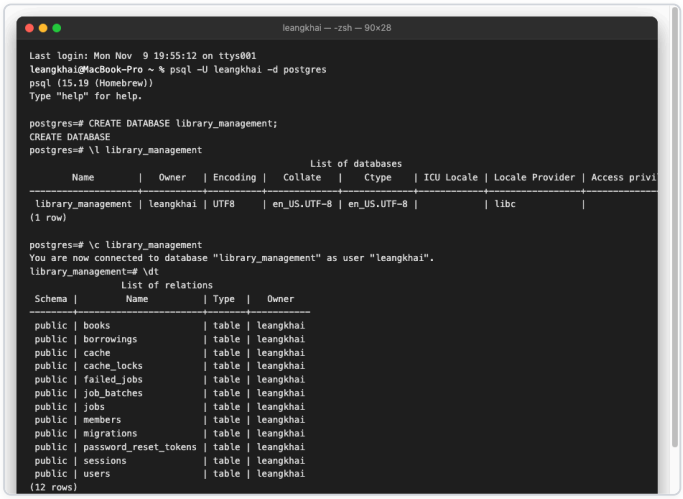


Figure 2: PostgreSQL terminal output verifying the library_management database and migrated tables.

4. Design an ER diagram for Books, Members, and Borrowings:

Below is the ER diagram for the Library Management System, showing the relationships between **Books**, **Members**, and **Borrowings**:

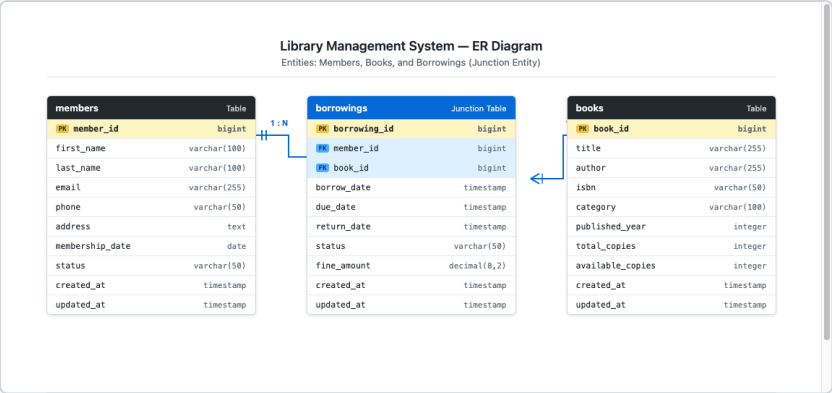


Figure 3: Relational ER diagram for Books, Members, and Borrowings.

Identify attributes and primary keys

Entity	Primary Key (PK)	Foreign Keys (FK)	Other Attributes
members	`member_id` (bigint)	—	first_name, last_name, email (unique), phone, address, membership_date, status
books	`book_id` (bigint)	—	title, author, isbn (unique), category, published_year, total_copies, available_copies
borrowings	`borrowing_id` (bigint)	`member_id`, `book_id`	borrow_date, due_date, return_date, status, fine_amount

Show relationships and cardinalities

- **Members to Borrowings (1 : 0..N):** One member can borrow zero, one, or multiple books over time. Each borrowing record belongs to exactly one member.
- **Books to Borrowings (1 : 0..N):** One book can be borrowed zero, one, or multiple times across its lifespan. Each borrowing transaction references exactly one book.
- **Members to Books (M : N):** Resolved through the `borrowings` junction table to maintain a clean, normalized relational schema.

Explain your design in 3–5 sentences

> I designed this library management database using three tables: Books, Members, and Borrowings. The Members table keeps track of user profiles, and the Books table holds book details and the number of copies in stock. Since one member can borrow multiple books over time, and one book can be borrowed by multiple members, this creates a many-to-many relationship. I resolved this by creating the Borrowings table as a junction table, which links Members and Books using foreign keys and records the borrow date, due date, return date, and any fines.